

step_1 CrossView4DVAE 模型架构与超参数手册

生成日期: 2026-05-27 适用目录: step_1

核心文件: crossview_vae.py, train_vae.py, data.py, losses.py, visual_debug.py

1. 一句话概览

CrossView4DVAE 是一个面向自动驾驶连续多视角视频的 4D VAE。输入来自 nuScenes / nus scene-folder 等数据源:

$x = [B, T, V_{cam}, 3, H, W]$, range $[-1, 1]$

当前版本的核心设计已经从“单层 cylinder + 先压时间再压视角”升级为:

物理相机视频

-> 两层 near/far cylinder 表示

-> near/far 不同时间/视角/空间压缩比

-> learned $T_q \times V_q \times H_q \times W_q$ joint cross attention 压缩

-> Gaussian latent

-> 两层 cylinder decoder

-> 反投影回物理相机做 loss

设计目标是: 利用自动驾驶视频中时间和多视角的冗余信息降低 latent token 数, 同时尽量保留近景物体、远景背景和跨相机连续性。

2. 符号约定

符号	含义
B	batch size
T	输入帧数, 例如 5
V_{cam}	物理相机数, nuScenes 通常是 6
V_v	virtual cylinder view 数, 即 virtual_view_count
D	cylinder layer 数, 当前推荐 D=2
r_{near}, r_{far}	near/far cylinder 半径, 例如 4m 和 20m
V_{near}	near layer latent view 数, 即 near_latent_view_count
V_{far}	far layer latent view 数, 即 far_latent_view_count
T_{near}	near layer latent time-query 数, 即 near_latent_time_count
T_{far}	far layer latent time-query 数, 即 far_latent_time_count
S_{near}	near layer latent spatial extra downsample, 即 near_latent_spatial_downsample_factor
S_{far}	far layer latent spatial extra downsample, 即 far_latent_spatial_downsample_factor
C0	base_channels
C_lat	latent_channels
H_l, W_l	latent 空间分辨率, 默认约为 H/8, W/8
H_{near}, W_{near}	near spatial query grid, $\text{ceil}(H_l/S_{near})$, $\text{ceil}(W_l/S_{near})$
H_{far}, W_{far}	far spatial query grid, $\text{ceil}(H_l/S_{far})$, $\text{ceil}(W_l/S_{far})$
N_{pack}	latent package 总 token 数, $T_{near} \times V_{near} \times H_{near} \times W_{near} +$ $T_{far} \times V_{far} \times H_{far} \times W_{far}$

推荐 debug 设置:

```
B=1, T=5, V_cam=6, V_v=6, D=2
near/far radii = [4, 20]
H=256, W=448
H_l=32, W_l=56
near: T_near=2, V_near=3, S_near=1 -> H_near=32, W_near=56
far : T_far=1, V_far=1, S_far=2 -> H_far=16, W_far=28
N_pack = 2*3*32*56 + 1*1*16*28 = 11200
C0=32, C_lat=16
```

对应 posterior:

```
mean/logvar: [B, 1, N_pack=11200, C_lat=16, 1, 1]
```

3. 总体数据流

```
input x [B,T,V_cam,3,H,W]
-> stem: physical-view independent causal 3D conv
-> project_layers: camera -> two-layer cylinder
    [B,T,D,V_v,C0,H,W]
-> flatten layers as batch: [B*D,T,V_v,C0,H,W]
-> down1 + view_mix1
-> down2 + view_mix2
-> down3 + view_mix3
-> pre_attn on each layer
-> unflatten layers: [B,T,D,V_v,4*C0,H_l,W_l]
-> LayeredTVCompressor
    near learned queries: [T_near,V_near,H_near,W_near]
    far learned queries: [T_far,V_far,H_far,W_far]
    concat -> [B,1,N_pack,4*C0,1,1]
-> to_moments -> Gaussian posterior
-> z = mode or sample
-> from_latent
-> LayeredTVExpander
    [B,T,D,V_v,4*C0,H_l,W_l]
-> flatten layers as batch
-> post_attn + up1 + view_mix_up1
-> up2 + view_mix_up2
-> up3 + head
-> decoded cylinder [B,T,D,V_v,3,H,W]
-> optional render_cylinder_to_cameras
    camera recon [B,T,V_cam,3,H,W]
```

当前 train_vae.py 在使用 --use-camera-params 时会启用完整闭环:

camera input -> layered cylinder latent -> layered cylinder output -> physical camera recon

4. 两层 Cylinder 表示

4.1 为什么从单层改为两层

单层 cylinder 相当于假设所有像素都落在同一个半径的曲面上。自动驾驶场景里这个假设很强:

内容	深度特征	单层问题
近处车辆、行人、杆子	视差大, 遮挡强	容易 ghosting 和边界错位

内容	深度特征	单层问题
远处建筑、树、天空 地面和车道线	跨相机更连续 不是垂直圆柱面	适合更强压缩 容易投影形变

两层表示保留 near/far 两个几何假设:

```
cylinder_radii = [4.0, 20.0]
near layer: r = 4m
far layer : r = 20m
```

这不是把多个半径平均掉, 而是保留独立 layer:

```
project_layers output = [B,T,D,V_v,C,H,W]
```

4.2 某个像素属于 near 还是 far

当前实现没有做 “原始相机像素 hard assignment 到 near/far”。它的方向是:

每个 cylinder layer 上的 3D 点

-> 投影到所有物理相机

-> grid_sample 得到该 layer 的颜色/特征

因此:

cylinder 像素属于哪一层: 由 layer index 决定

相机像素属于哪一层: 当前没有显式唯一归属

一个物理相机像素可能被 near layer 和 far layer 的不同 cylinder 点采样到。最终 decoder 和 renderer 学习如何组合 near/far 信息。

如果后续需要显式判断某个相机像素更依赖 near 还是 far, 可以增加 learned layer weight:

```
w_near(p), w_far(p), w_near + w_far = 1
I_camera(p) = w_near(p) * sample(near) + w_far(p) * sample(far)
```

那时 visual debug 可以保存:

```
layer_assignment_near.png
layer_assignment_far.png
```

5. 相机参数如何进入模型

训练入口:

```
K = batch["camera_intrinsics"]
E = batch["camera_transforms"]
intr_hw = batch["intrinsics_hw"]
output = model(x, intrinsics=K, extrinsics=E, intrinsics_hw=intr_hw)
```

模型内部:

```
CrossView4DVAE.forward()
-> encode()
-> _encode_trainable_body()
-> h = stem(x)
-> h = virtual_projector.project_layers(h, K, E, intr_hw)
```

相机参数只在 projector/renderer 中使用:

模块	是否使用 K/E	说明
stem	否	每个物理相机独立提特征
CylindricalViewProjector.project_layers	是	camera -> near/far cylinder
render_cylinder_to_cameras	是	near/far cylinder -> physical camera
ViewMixResBlock	否	在规则 cylinder view 轴上做 circular conv
TVJointViewCompressor	否	用规则 T/V/H 坐标做 attention
RoPE3DBottleneckAttention	否	规则 cylinder grid RoPE

相机参数的作用是把数据映射到统一的 canonical cylinder 空间，而不是直接作为 RoPE 坐标输入。

6. Encoder 逐层架构

6.1 stem

```
self.stem = CausalConv3d(3 -> C0, kernel=3, padding=1)
```

数据变化:

```
[B,T,V_cam,3,H,W]
-> reshape [B*V_cam,3,T,H,W]
-> causal 3D conv
-> [B,T,V_cam,C0,H,W]
```

性质:

- 时间卷积是 causal。
- 每个物理相机独立处理。
- stem 不混合视角。

6.2 project_layers

```
self.virtual_projector.project_layers(h, K, E, intr_hw)
```

输入输出:

```
input : [B,T,V_cam,C0,H,W]
output: [B,T,D,V_v,C0,H,W]
```

对每个 radius 独立执行:

```
for r in cylinder_radii:
    build cylinder 3D points
    transform ego -> camera
    project with K
    grid_sample source cameras
    soft blend by valid, angle, image-border weights
```

6.3 Layer flatten

后续主干卷积仍复用原有 [B,T,V,C,H,W] 结构，因此把 layer 临时展到 batch:

```
[B,T,D,V_v,C0,H,W]
-> [B*D,T,V_v,C0,H,W]
```

这样 near/far 两层共享 encoder 权重，但不会被直接平均掉。

6.4 down1 + view_mix1

down1: $C_0 \rightarrow C_0$, stride_t=1, stride_{hw}=2
view_mix1: CircularConv3d over view axis

数据变化:

$[B \times D, T, V_v, C_0, H, W]$
 $\rightarrow [B \times D, T, V_v, C_0, H/2, W/2]$

ViewMixResBlock 用 circular padding, 所以第一个 view 和最后一个 view 相邻。

6.5 down2 + view_mix2

down2: $C_0 \rightarrow 2 \times C_0$, stride_t=1, stride_{hw}=2

当 $T=5$:

$T=5 \rightarrow 5$
 $H, W \rightarrow H/4, W/4$

这里不再用卷积 stride 压缩时间; 时间 token 会在后面的 learned T/V query compressor 里和视角一起压缩。

6.6 down3 + view_mix3

down3: $2 \times C_0 \rightarrow 4 \times C_0$
stride_t = 1
stride_{hw} = 2

默认设置下:

$T=5 \rightarrow 5$
 $H, W \rightarrow H/8, W/8$

temporal_downsample_factor 目前保留为旧命令兼容和默认预算参考, 不再驱动 encoder 的时间 stride。

6.7 pre_attn

RoPE3DBottleneckAttention($4 \times C_0$, num_attention_heads)

作用:

- 在每一层 cylinder 内做 cross-view + spatial attention。
- token 化为 $[B \times D \times T, V_v \times H_1 \times W_1, C]$ 。
- 使用规则 cylinder grid 的 azimuth/height RoPE。

6.8 LayeredTVCompressor

这是当前架构的核心压缩点:

input : $[B, T, D, V_v, 4 \times C_0, H_1, W_1]$
output: $[B, 1, N_{\text{pack}}, 4 \times C_0, 1, 1]$

每层独立设置 latent time/view/spatial budget:

near: $[T, V_v, H_1, W_1] \rightarrow [T_{\text{near}}, V_{\text{near}}, H_{\text{near}}, W_{\text{near}}]$
far : $[T, V_v, H_1, W_1] \rightarrow [T_{\text{far}}, V_{\text{far}}, H_{\text{far}}, W_{\text{far}}]$
concat: $N_{\text{pack}} =$
 $T_{\text{near}} \times V_{\text{near}} \times H_{\text{near}} \times W_{\text{near}}$
 + $T_{\text{far}} \times V_{\text{far}} \times H_{\text{far}} \times W_{\text{far}}$

例如:

```

T=5, V_v=6, D=2, H_l=32, W_l=56
near: T_near=2, V_near=3, S_near=1 -> 2*3*32*56 = 10752 tokens
far : T_far=1, V_far=1, S_far=2 -> 1*1*16*28 = 448 tokens
source tokens = D * T * V_v * H_l * W_l = 107520
latent tokens = 11200
total token compression ratio ~= 9.6x

```

7. T-V-S Joint Cross Attention 是否真的存在

当前实现中, TVJointViewCompressor 的 key/value token 是整个时间、视角和空间网格:

```

q_tokens : [B, T_q * V_q * H_q * W_q, C]
kv_tokens: [B, T   * V_v * H_l * W_l, C]

```

也就是说, 每个 latent query 可以 attend 到同一 layer 内所有时间、所有视角、所有空间位置的 source token。

RoPE 坐标包含:

```

time coordinate
azimuth coordinate
height coordinate

```

因此当前版本已经不是旧的 “每个时间帧独立压 view”, 而是:

learned T-V-S joint attention compression

这次修改后, 时间 token 数也由 learned query 决定, 而不是由前面的 temporal stride conv 决定。down2/down3 保持 stride_t=1, 源端保留完整 T, 然后每个 layer 用自己的 query grid 同时选择时间和视角信息:

```

near: T_near x V_near
far : T_far  x V_far

```

空间维度同理由每层自己的 S 决定:

```

near: H_near = ceil(H_l / S_near), W_near = ceil(W_l / S_near)
far : H_far  = ceil(H_l / S_far),  W_far  = ceil(W_l / S_far)

```

8. Gaussian Latent

to_moments:

Linear(4*C0 -> 2*C_lat)

数据变化:

```

[B,1,N_pack,4*C0,1,1]
-> mean/logvar [B,1,N_pack,C_lat,1,1]

```

posterior:

$q(z|x) = N(\text{mean}, \exp(\text{logvar}))$

训练默认 deterministic:

`z = posterior.mode() = mean`

只有显式加:

`--sample-posterior`

才使用 stochastic sampling。

9. Decoder 逐层架构

9.1 from_latent

Linear(C_lat -> 4*C0)

z [B,1,N_pack,C_lat,1,1]
-> [B,1,N_pack,4*C0,1,1]

9.2 LayeredTVExpander

输入 latent package 会按 layer budget 切开:

near part: [B,T_near*V_near*H_near*W_near,4*C0,1,1]
far part : [B,T_far*V_far*H_far*W_far, 4*C0,1,1]

然后分别 reshape 成 [T_q,V_q,H_q,W_q] query grid, 再用 cross attention 展开到完整时间、virtual views 和 latent feature 空间:

output: [B,T,D,V_v,4*C0,H_l,W_l]

9.3 Layer flatten + decoder upsample

[B,T,D,V_v,4*C0,H_l,W_l]
-> [B*D,T,V_v,4*C0,H_l,W_l]
-> post_attn
-> up1 + view_mix_up1
-> up2 + view_mix_up2
-> up3 + head
-> [B*D,T,V_v,3,H,W]
-> unflatten [B,T,D,V_v,3,H,W]

最终 cylinder 输出:

pred_cylinder: [B,T,D,V_v,3,H,W]

如果 render_camera=True 且启用 camera params:

sample: [B,T,V_cam,3,H,W]

否则 sample 是 layer 平均后的 cylinder view:

sample: [B,T,V_v,3,H,W]

训练真实 nuScenes 时推荐使用 camera params 和 camera render 闭环。

10. 典型 Shape 表

设置:

B=1, T=5, V_cam=6, V_v=6, D=2
H=256, W=448
C0=32, C_lat=16
T_near=2, V_near=3
T_far=1, V_far=1
S_near=1, S_far=2
H_near=32, W_near=56
H_far=16, W_far=28
N_pack=11200
cylinder_radii=[4,20]

阶段	输出形状	说明
input	[1,5,6,3,256,448]	原始多相机视频
stem	[1,5,6,32,256,448]	每个物理 view 独立提特征
project_layers	[1,5,2,6,32,256,448]	near/far 两层 cylinder
flatten layers	[2,5,6,32,256,448]	layer 进 batch
down1	[2,5,6,32,128,224]	空间 /2
down2	[2,5,6,64,64,112]	空间 /2, 时间不压
down3	[2,5,6,128,32,56]	空间 /2, 时间不压
pre_attn	[2,5,6,128,32,56]	每层内 view/spatial attention
unflatten	[1,5,2,6,128,32,56]	恢复 layer 维度
LayeredTVCompressor	[1,1,11200,128,1,1]	near 2x3x32x56 + far 1x1x16x28
mean/logvar	[1,1,11200,16,1,1]	posterior
LayeredTVExpander	[1,5,2,6,128,32,56]	恢复完整时间和两层 6 virtual views
up1	[2,5,6,64,64,112]	layer 再展 batch, 空间上采样
up2	[2,5,6,32,128,224]	空间上采样
up3/head	[2,5,6,3,256,448]	RGB cylinder
unflatten	[1,5,2,6,3,256,448]	decoded cylinder
render camera	[1,5,6,3,256,448]	physical camera recon

11. Loss 结构

当前训练 loss:

```
loss =
    camera_loss_weight * camera_rec
+ cylinder_loss_weight * cylinder_rec
+ camera_loss_weight * perceptual_weight * perceptual
+ camera_loss_weight * edge_loss_weight * camera_edge
+ cylinder_loss_weight * cylinder_edge_loss_weight * cylinder_edge
+ kl_weight_now * KL
+ logvar_reg_weight * logvar_reg
+ cylinder_seam_weight * seam_loss
```

默认情况下 perceptual_weight、edge_loss_weight、cylinder_edge_loss_weight 都是 0，因此新增项不会改变旧训练命令的行为。它们主要用于单样本 overfit 或弱瓶颈实验中判断：糊是来自信息瓶颈，还是来自纯 L1 目标偏向低频平均。

11.1 cylinder_rec

```
target_cylinder = virtual_projector.project_layers(x, K, E, intr_hw)
pred_cylinder = output["cylinder_sample"]
```

形状:

[B,T,D,V_v,3,H,W]

如果启用:

```
--masked-cylinder-loss
```

则只在有效覆盖区域监督:

```
cylinder_mask = project_layers(ones_like_input)
loss only where cylinder_mask > cylinder_mask_threshold
```

作用:

- 让模型先学干净 canonical two-layer cylinder。
- 避免无效投影区域的灰色/零值污染训练。

11.2 camera_rec

```
pred_cylinder
-> render_cylinder_to_cameras(K, E)
-> recon_camera [B,T,V_cam,3,H,W]
-> compare with x
```

作用:

- 保留物理相机语义的最终约束。
- 防止 cylinder 表示自己看起来合理, 但反投影回真实相机后不对。

推荐早期设置:

```
--reconstruction-domain both
--masked-cylinder-loss
--cylinder-loss-weight 1.0
--camera-loss-weight 0.2
```

含义:

主监督: two-layer cylinder 重建

辅助监督: 物理相机重建

11.3 perceptual

感知损失只建议放在 camera-domain:

```
perceptual = LPIPS(recon_camera, x)
```

原因是 LPIPS 的预训练语义来自自然 RGB 图像, 而 camera-domain 正好是最终物理相机画面。cylinder-domain 是多相机几何投影后的 canonical 表示, 包含 overlap、mask、near/far 固定半径和 投影边界; 直接在 cylinder 上使用 LPIPS 容易把 projector 伪影当成纹理错误来惩罚。

实现细节:

- 输入 [B,T,V,3,H,W] 会 flatten 为 [B*T*V,3,H,W]。
- 通过 --perceptual-batch-size 分块计算 LPIPS, 避免一次性把所有帧/视角塞进感知网络。
- 如果环境没有安装 lpips, 代码会自动退回到 0 感知损失并打印提示。

推荐起步:

```
--perceptual-weight 0.03
--perceptual-batch-size 2
```

11.4 camera_edge 与 cylinder_edge

edge loss 是一阶空间梯度 L1, 用来惩罚高频边缘被平均掉:

```
dx = image[..., :, 1:] - image[..., :, :-1]
dy = image[..., 1:, :] - image[..., :-1, :]
```

```
edge(pred, target) =
    0.5 * (L1(dx_pred, dx_target) + L1(dy_pred, dy_target))
```

camera-domain:

```
camera_edge = edge(recon_camera, x)
```

cylinder-domain:

```
cylinder_edge = edge(pred_cylinder, target_cylinder)
```

如果启用 `--masked-cylinder-loss`, `cylinder_edge` 也使用同一份有效区域 `mask`; 只有相邻两个像素都有效时, 该方向的梯度才参与 `loss`。这样可以避免无效 `cylinder` 区域的 灰色/零值边界反过来教模型生成假边缘。

推荐起步:

```
--edge-loss-weight 0.1
--cylinder-edge-loss-weight 0.05
```

11.5 KL 与 posterior 正则

```
KL = posterior.kl()
weighted_kl = kl_weight_now * KL
```

KL 是对所有 latent 元素求和后按 batch 平均, 因此 raw KL 可能很大。判断训练时应看:

```
rec / cylinder_rec / camera_rec / camera_edge / cylinder_edge / perceptual / weighted_kl
```

12. 关键超参数

12.1 架构参数

参数	推荐值	说明
<code>--base-channels</code>	32 或 64	主干容量, 越大越清晰但更慢
<code>--latent-channels</code>	16	latent channel 数, 太小容易糊
<code>--virtual-view-count</code>	6	与 nuScenes 6 相机对齐
<code>--cylinder-radii</code>	4 20	near/far 两层 cylinder
<code>--latent-time-count</code>	2	旧命令兼容和默认 time-query 数
<code>--near-latent-time-count</code>	2	near layer 保留更多时间 token
<code>--far-latent-time-count</code>	1	far layer 时间压缩更强
<code>--near-latent-view-count</code>	3	near layer 保留更多视角 token
<code>--far-latent-view-count</code>	1	far layer 更强压缩
<code>--latent-spatial-downsample-factor</code>	1	默认额外空间压缩倍率
<code>--near-latent-spatial-downsample-factor</code>	1	near layer 保留完整 latent feature 空间
<code>--far-latent-spatial-downsample-factor</code>	2	far layer 空间 token 再压 2x
<code>--latent-view-count</code>	可省略	仅作为旧命令兼容和默认值
<code>--temporal-downsample-factor</code>	可省略	旧命令兼容; 当前 encoder 不再用它做时间 stride
<code>--num-attention-heads</code>	2	注意力头数
<code>--num-bottleneck-blocks</code>	1	pre/post attention 层数

12.2 Loss 参数

参数	推荐值	说明
<code>--reconstruction-domain</code>	both	同时使用 cylinder 和 camera loss
<code>--masked-cylinder-loss</code>	on	排除无效 cylinder 区域
<code>--cylinder-loss-weight</code>	1.0	cylinder 主监督
<code>--camera-loss-weight</code>	0.2	camera 辅助监督
<code>--kl-weight</code>	1e-7 或 1e-6	overfit 阶段建议小
<code>--kl-warmup</code>	3000	先重建, 再逐步加入 KL
<code>--perceptual-weight</code>	0.03 或 0	camera-domain LPIPS; debug 初期可先关, 糊图排查时打开
<code>--perceptual-batch-size</code>	2 或 4	LPIPS 分块大小, 越小越省显存
<code>--edge-loss-weight</code>	0.1 或 0	camera-domain 一阶梯度 loss

参数	推荐值	说明
--cylinder-edge-loss-weight	0.05 或 0	cylinder-domain 一阶梯度 loss, 启用 mask 时只监督有效区域
--logvar-reg-weight	0	需要稳定 posterior 时再开
--cylinder-seam-weight	0 或 小值	seam 正则, 先 debug 后再开

12.3 数据与运行参数

参数	推荐值	说明
--data	nuscenes_scene	读取 scene-folder
--image-hw	256 448	当前 nuScenes debug 分辨率
--sequence-length	5	连续视频长度
--batch-size	1	单样本 overfit
--num-workers	0	debug 阶段最稳
--amp	bf16	服务器上推荐
--use-camera-params	on	启用 two-layer cylinder 闭环
--ego-coordinate-mode	nuscenes	nuScenes 坐标系

13. 推荐训练命令

13.1 单样本 overfit

```
CUDA_VISIBLE_DEVICES=0 python -m step_1.train_vae \
  --data nuscenes_scene \
  --nusc-data-root /shareNFS_40/sharedata/nuscenes/nuscenes_trainval \
  --nusc-split train \
  --sequence-length 5 \
  --image-hw 256 448 \
  --batch-size 1 \
  --num-workers 0 \
  --overfit-samples 1 \
  --overfit-sample-index 1 \
  --base-channels 32 \
  --latent-channels 16 \
  --virtual-view-count 6 \
  --latent-time-count 2 \
  --near-latent-time-count 2 \
  --near-latent-view-count 3 \
  --near-latent-spatial-downsample-factor 1 \
  --far-latent-time-count 1 \
  --far-latent-view-count 1 \
  --far-latent-spatial-downsample-factor 2 \
  --cylinder-radii 4 20 \
  --cylinder-height-scale 1.4 \
  --use-camera-params \
  --ego-coordinate-mode nuscenes \
  --reconstruction-domain both \
  --masked-cylinder-loss \
  --cylinder-loss-weight 1.0 \
  --camera-loss-weight 0.2 \
  --edge-loss-weight 0.1 \
  --cylinder-edge-loss-weight 0.05 \
```

```

--perceptual-weight 0.03 \
--perceptual-batch-size 2 \
--steps 3000 \
--lr 5e-4 \
--warmup-steps 100 \
--kl-weight 1e-7 \
--kl-warmup 3000 \
--log-every 10 \
--preview-every 100 \
--ckpt-every 500 \
--amp bf16 \
--out step_1/runs/nus_overfit_2layer_joint_tvs_near2x3s1_far1x1s2

```

13.2 Visual debug

```

python -m step_1.visual_debug \
--data nusenes_scene \
--nusc-data-root /shareNFS_40/sharedata/nuscenes/nuscenes_trainval \
--nusc-split train \
--sample-index 1 \
--sequence-length 5 \
--image-hw 256 448 \
--base-channels 32 \
--latent-channels 16 \
--virtual-view-count 6 \
--latent-time-count 2 \
--near-latent-time-count 2 \
--near-latent-view-count 3 \
--near-latent-spatial-downsample-factor 1 \
--far-latent-time-count 1 \
--far-latent-view-count 1 \
--far-latent-spatial-downsample-factor 2 \
--cylinder-radii 4 20 \
--cylinder-height-scale 1.4 \
--use-camera-params \
--ego-coordinate-mode nusenes \
--checkpoint step_1/runs/nus_overfit_2layer_joint_tvs_near2x3s1_far1x1s2/ckpts/last.pt \
--out step_1/runs/visual_debug_2layer_joint_tvs_nus \
--device cuda

```

14. Visual Debug 如何判断问题

新版 visual_debug.py 会输出:

```

input/
projector/
projector_roundtrip/
cylinder/
latent/
reconstruction/
activations/
rope/
debug_metrics.json
report.md

```

14.1 判断 projector / renderer 是否有问题

优先看:

```
projector_roundtrip/target_render_camera.png
projector_roundtrip/target_render_abs_error.png
projector_roundtrip/camera_render_coverage.png
```

如果这里已经有明显灰区、竖线或高误差, 说明问题在:

camera -> cylinder -> camera

还没有进入 VAE, 不应先怪 latent 压缩。

14.2 判断 cylinder target 是否有 seam

看:

```
cylinder/target_cylinder_grid_00_near_r4.png
cylinder/target_cylinder_grid_01_far_r20.png
cylinder/target_seams_*.png
```

如果 target seam 强, 说明 projector 生成的训练目标本身不连续。

14.3 判断 decoder 是否制造 seam

对比:

```
cylinder/target_seams_*.png
cylinder/pred_seams_*.png
cylinder/error_seams_*.png
```

如果 target seam 弱但 pred/error seam 强, 问题在 decoder 或 latent package 展开。

14.4 判断 near/far 压缩是否过强

看:

```
latent/view_down_energy_00_near_r4.png
latent/view_down_energy_01_far_r20.png
latent/posterior_mean_energy_*.png
debug_metrics.json -> latent -> budget
```

budget 会记录:

```
source_time_tokens
virtual_views_per_layer
layer_latent_time_view_counts
layer_latent_spatial_downsample_factors
layer_latent_spatial_shapes
layer_latent_time_view_token_counts
layer_latent_token_counts
total_source_time_view_tokens
total_latent_time_view_tokens
compression_ratio_time_view_axis
total_source_tokens_including_spatial
total_latent_tokens_including_spatial
compression_ratio_total_token_axis
```

其中 compression_ratio_time_view_axis 只看 T/V 轴, compression_ratio_total_token_axis 才包含空间 token。判断模型是否真的加速, 优先看后者。

如果 near energy 塌缩、近景重建糊，说明 near 的 T/V/S token budget 过强。可以尝试：

```
--near-latent-time-count 3
--near-latent-view-count 4
--near-latent-spatial-downsample-factor 1
```

如果 far 层几乎无贡献，可尝试：

```
--far-latent-time-count 2
--far-latent-view-count 2
--far-latent-spatial-downsample-factor 1
```

14.5 判断模型整体是否欠拟合

如果：

```
projector_roundtrip 正常
target cylinder 正常
latent energy 正常
但 recon 仍然糊
```

优先增加：

```
--latent-channels 16 或 32
--base-channels 64
```

或者降低空间压缩强度，但这需要继续改主干结构。

15. 当前实现的已知边界

1. 当前 T/V/S 压缩是 per-layer 的：near 和 far 各自用 learned query 压缩，还没有跨 layer 的 token routing 或 cross-layer attention。
2. 当前 near/far layer 没有显式像素级 assignment，物理相机像素不会被 hard 分配到某一层。
3. 两层 cylinder 仍不能完美表达地面、天空和复杂非圆柱几何，后续可考虑 layer weight、depth-aware composition 或更多几何先验。
4. latent-time-count、latent-view-count 和 latent-spatial-downsample-factor 仍保留用于兼容旧命令和默认预算。当前两层实验应优先使用 near/far-latent-time-count、near/far-latent-view-count 与 near/far-latent-spatial-downsample-factor。
5. temporal_downsample_factor 目前保留用于旧命令兼容和默认预算参考，不再控制 encoder temporal stride。
6. cylinder_seam_weight 目前建议先关闭，等 projector target 干净后再作为正则加入。

16. 当前阶段结论

当前 step_1 的研究假设可以概括为：

自动驾驶视频的多视角冗余不是均匀分布的。

远景跨相机重复更多，适合强压缩；

近景视差和遮挡更强，应该弱压缩。

因此模型采用：

```
two-layer cylinder geometry
+ near/far non-uniform T-V-S token budget
+ T-V-S joint cross attention compression
+ physical camera reconstruction loss
```

后续实验应重点比较：

实验	目的
单层 vs 两层 cylinder	验证几何表达是否改善
near/far = 2x3s1 / 1x1s2 vs 2x3s1 / 1x1s1	验证非均匀空间压缩是否有效
near/far = 2x3 / 1x1 vs 2x3 / 2x2	验证非均匀 T/V 压缩是否有效
time budget 1/2/3	判断时间压缩是否导致运动信息丢失
camera loss 0.0 vs 0.2	验证物理相机语义约束是否必要
no compression vs near/far compression	判断糊是否来自 token budget
projector_roundtrip 指标	判断灰区/竖线是否来自几何闭环